Classification de Nuages de Points 3D : Graph CNN, Apprentissage Fédéré et Distillation de Connaissances

26 mai 2026

Note : le rendu de ce projet est fixé au **jour de l'examen**.

1 Contexte

Ce projet porte sur la classification d'objets 3D représentés sous forme de nuages de points. L'objectif est de comparer trois stratégies d'entraînement sur des architectures de réseaux de neurones sur graphes :

— une **approche centralisée**, qui sert de référence,

— une **approche fédérée**, où les données ne quittent jamais les clients,

— une approche par **distillation de connaissances**, où un modèle léger est guidé par le modèle centralisé.

Chaque binôme travaille sur **deux architectures Graph CNN distinctes**, choisies librement. La seule contrainte est que **la combinaison choisie soit unique dans la promotion** : aucun autre binôme ne peut travailler sur la même paire. Les binômes déclarent leur choix à l'enseignant selon le principe premier arrivé, premier servi.

2 Choix des architectures

Chaque binôme choisit librement deux architectures Graph CNN et les déclare à l'enseignant. La seule contrainte est que **la paire soit unique dans la promotion** : premier arrivé, premier servi.

3 Données

Le jeu de données est **ShapeNet via CAP3D**, disponible à l'adresse :

`https://huggingface.co/datasets/tiange/Cap3D/tree/main/PointCloud_zips_ShapeNet`

Il contient **52 476 objets 3D** répartis sur **51 classes**, pour un volume total de **27 Go**. Chaque objet est un fichier `.ply` contenant jusqu'à 16 384 points, chaque point portant des coordonnées spatiales (x, y, z) et des valeurs de couleur (r, g, b).

Le fichier `labeled_dataset.csv` fournit la correspondance entre les noms de fichiers `.ply` et leurs étiquettes de classe réelles. C'est le point d'entrée pour construire le dataset : chaque ligne associe un nom de fichier à sa classe, et c'est à partir de ce fichier que les partitions (entraînement, validation, test) doivent être construites.

Étant donné la taille du dataset, les étudiants travaillent sur un **sous-ensemble représentatif**. Le choix du sous-ensemble doit respecter les bonnes pratiques : équilibre entre les classes, séparation stricte entraînement / validation / test sans fuite de données, et reproductibilité (graine aléatoire fixée).

Le pipeline de données est à définir et implémenter librement. Il doit au minimum couvrir le chargement des fichiers `.ply`, le sous-échantillonnage à un nombre fixe de points, la normalisation, et les augmentations à l'entraînement uniquement.

4 Travail demandé

4.1 Partie 1 — Baseline centralisée

Entraîner chacune des deux architectures sur l'ensemble des données d'entraînement. Cette configuration sert de référence pour toute la comparaison.

4.2 Partie 2 — Apprentissage fédéré (implémentation manuelle)

Règle absolue : les mécanismes de fédération doivent être entièrement codés à la main. Toute bibliothèque tierce gérant automatiquement l'agrégation, la synchronisation des poids ou la coordination des clients est **interdite** et entraîne une note de zéro sur cette partie.

La communication entre les clients et le serveur s'effectue **uniquement** par échange de `state_dict` PyTorch. Concrètement :

— Le **serveur** diffuse une copie du modèle global à chaque client (`copy.deepcopy` obligatoire), collecte les `state_dict` mis à jour après entraînement local, et applique l'agrégation.

— Chaque **client** charge le modèle reçu, s'entraîne sur ses données privées, et renvoie son `state_dict` mis à jour ainsi que la taille de son jeu de données local.

— **L'agrégation** (FedAvg) est une moyenne pondérée des paramètres par la taille de chaque jeu de données local :

$$\mathbf{w}^{(t+1)} = \sum_{k=1}^{K} \frac{n_k}{n} \mathbf{w}_k^{(t)}, \qquad n = \sum_{k=1}^{K} n_k \tag{1}$$

Elle doit être implémentée explicitement, clé par clé dans le `state_dict`, en distinguant les paramètres flottants des buffers entiers (e.g. BatchNorm).

Ce pipeline est implémenté pour les deux architectures dans les deux modes suivants.

Fédération horizontale (HFL) — partition par domaine sémantique

Les 51 classes du dataset sont réparties entre 4 clients. Chaque client possède des objets de catégories différentes, simulant des agents spécialisés qui n'ont jamais vu les classes des autres.

Client	Classes assignées
Client 1 — Mobilier	table, chair, bed, sofa, bench, bookshelf, file cabinet, cabinet
Client 2 — Transport	car, airplane, train, bus, motorbike, bicycle, watercraft, rocket
Client 3 — Technologie	laptop, telephone, camera, earphone, keyboard, display, printer, microphone, remote
Client 4 — Domestique	bottle, bowl, mug, can, knife, jar, basket, rifle, pistol, lamp, faucet, stove, washer, dishwasher, trash bin, mailbox, birdhouse, bag, cap, guitar, piano

Chaque client entraîne son modèle local sur ses classes uniquement, puis contribue à l'agrégation FedAvg. L'analyse doit inclure une évaluation du modèle global sur des classes dont le client responsable était absent ou sous-représenté lors d'une ronde.

Fédération verticale (VFL) — partition par attributs

Ici ce ne sont pas les objets qui sont divisés entre les entités, mais **l'information portée par chaque point**. Chaque objet est connu des deux entités, mais sous des angles différents.

— **Entité A — Géométrie :** possède uniquement les coordonnées spatiales (x, y, z) de chaque point (`pos` dans le tenseur PyTorch). Elle voit la structure pure de l'objet, sans aucune information de couleur.

— **Entité B — Apparence :** possède uniquement les valeurs de couleur (r, g, b) associées à chaque point (`colors` dans le tenseur PyTorch). Elle voit l'apparence de l'objet, sans aucune information géométrique.

Le flux d'entraînement est le suivant :

1. **Encodage local :** l'Entité A traite les coordonnées XYZ via son tronc de réseau et produit un vecteur d'embedding $\mathbf{e}_A \in \mathbb{R}^d$. L'Entité B fait de même avec les couleurs RGB et produit $\mathbf{e}_B \in \mathbb{R}^d$.

2. **Envoi au serveur :** les deux entités transmettent leurs embeddings au serveur central. **Aucune entité n'envoie ses données brutes** — seuls les vecteurs intermédiaires circulent.

3. **Fusion et classification :** le serveur concatène $[\mathbf{e}_A \,\|\, \mathbf{e}_B]$ et passe le vecteur résultant dans une couche fully-connected pour produire la prédiction finale.

4. **Rétropropagation :** le serveur calcule la perte, puis renvoie les gradients à chaque entité. Chaque entité met à jour ses propres poids sans jamais accéder aux données ou aux poids de l'autre.

Les étudiants doivent justifier dans leur rapport pourquoi ce protocole garantit la confidentialité des données de chaque entité, et décrire la conception de la couche de fusion côté serveur.

4.3 Partie 3 — Distillation de connaissances

Pour chaque architecture, le modèle centralisé (Partie 1) joue le rôle d'enseignant. Un modèle allégé de la même famille (au moins 4 fois moins de paramètres) joue le rôle d'élève. La fonction de perte est :

$$\mathcal{L} = (1-\alpha)\,\mathcal{L}_{\mathrm{CE}}(\hat{y}, y) + \alpha\,\tau^2\,D_{\mathrm{KL}}\Big(\sigma\big(\tfrac{z_s}{\tau}\big) \;\Big\|\; \sigma\big(\tfrac{z_t}{\tau}\big)\Big) \tag{2}$$

avec τ la température et α le coefficient de pondération. Les logits de l'enseignant doivent être détachés du graphe de calcul. L'impact de τ et de α sur les performances doit être analysé.

5 Comparaison et analyse

Pour chacune des deux architectures, les quatre configurations suivantes doivent être évaluées et comparées sur le jeu de test :

ID	Configuration	Description
C1	Baseline centralisée	Toutes les données
C2	Fédéré horizontal IID	FedAvg, données IID
C3	Fédéré horizontal non-IID	FedAvg, données non-IID
C4	Distillation	Modèle élève guidé par C1

La comparaison doit inclure au minimum : la précision globale, la précision moyenne par classe, le nombre de paramètres, et les courbes de convergence.

Le rapport doit également comparer **les deux architectures entre elles** sur l'ensemble de ces configurations, avec une discussion critique des différences observées.

6 Livrable

Un seul livrable : un **dépôt contenant le code et un rapport PDF**.

Le code doit être propre, commenté et documenté — la structure interne est laissée à la discrétion du binôme.

Le rapport doit couvrir : le choix et la description des architectures, le pipeline fédéré implémenté, la distillation, les résultats comparatifs, et une conclusion.

7 Conclusion

À l'issue de ce projet, vous aurez conçu et implémenté de bout en bout un pipeline complet de classification 3D distribué. Vous aurez manipulé directement les paramètres de modèles PyTorch pour simuler un système fédéré réel, compressé un modèle par distillation, et conduit une analyse comparative rigoureuse entre plusieurs paradigmes d'entraînement et plusieurs architectures. Ce travail vous confronte aux contraintes concrètes du déploiement de modèles dans des environnements distribués et à ressources limitées.